

How the clearance queue works (explained simply)

The big picture: a bakery with numbered tickets

Imagine a bakery that sells fresh bread. Everyone who wants bread takes a numbered ticket. When bread comes out of the oven, whoever has the **lowest ticket number** gets served first. That's the whole idea. In this system:

- "Bread" = stock (products in the warehouse).
- "Taking a ticket" = an order getting its **clearance timestamp** (`clearance_date`).
- "Getting served" = the order's line actually holding reserved stock `stock.move` state = `assigned`).

When do you get a ticket?

You don't get a ticket just for walking into the bakery (creating an order). You get one the moment you **actually pay** (or, during a short grace period right after confirming, before payment even lands — like a tab you're expected to settle soon). No payment, no ticket, no bread — ever. That's `fulfillment_stage == "no_invoice"` : zero claim on stock, full stop.

Not everyone is in the same line — there are VIP lines too

Most people are in the ordinary line, sorted purely by ticket number (earliest timestamp wins). But there are two special lines that cut ahead of the ordinary line entirely:

1. **Force-Reserved** (an admin manually says "give this order stock NOW") — jumps ahead of every ordinary ticket-holder, but *only* for bread that's actually sitting on the shelf unclaimed. It can't take bread out of someone else's hands.
2. **Hard Lock** — this one's different. It's not a line at all. It's a bakery rule: "once you hand this person their loaf, NEVER take it back, no matter what." It gives zero help getting NEW bread — it only protects bread you already have.

So the real pecking order for *new* bread as it comes out of the oven is:

1. Scheduled Future Stock holders reclaiming a promised loaf (explained below) — top priority.
2. Force-Reserved.
3. Ordinary ticket-holders, earliest timestamp first.
4. Anyone with no ticket at all (`no_invoice`) — never served.

How the engine actually decides, every time something changes

Every time *anything* relevant happens — someone pays, an admin locks an order, a delivery truck arrives — the whole bakery does this:

1. **Take back every loaf that isn't protected.** Anything held by an ordinary ticket-holder or a foreign (unpaid) holder gets put back on the shelf. Hard-locked and force-reserved loaves are never touched.
2. **Hand the shelf back out, ticket by ticket, in the priority order above.** So if someone with an

earlier ticket shows up *after* a later-ticket person already grabbed a loaf, the earlier person steals it back on the next round. Nobody has a "permanent" claim just because they got there first chronologically — the *ticket number* rules, not *arrival order*.

This "release everything, then redeal fairly" approach is why the queue is always self-correcting: it doesn't matter what random order things happened in real life, the shelf always ends up matching the ticket-number order.

"Scheduled Future Stock" — the smart trade

Here's a clever bit. Say Order A has bread today but doesn't actually need it for 3 months. Order B has an earlier need date and there's a truck bringing more bread in 2 weeks. Instead of Order B just waiting empty-handed for those 2 weeks, the bakery says to Order A:

"Hey, you don't need this yet. Give your loaf to Order B now, and we PROMISE you the very first loaf off the next truck instead."

Order A hands it over, and gets flagged with a special "reclaim priority" — higher than literally anything else — so when that truck arrives, Order A gets first dibs on it, ahead of even a brand-new force-reserve. This only happens when there's a real, committed truck (a confirmed Purchase Order) coming with enough margin:

- **7 days** early if the shipment's logistics are confirmed (container reference + port arrival date both on file).
- **30 days** early if it's still just a planned date, not locked-down real logistics data.

"Scheduled Far Out" — you don't even get a shelf spot

If your own need-date is more than 6 months away, the bakery doesn't let you compete for TODAY's bread at all — you're not even given a spot in line, real ticket or not. You could have the earliest ticket in the whole bakery and it wouldn't matter; you're excluded until your date gets closer. This stops someone who won't need bread until next year from hoarding stock that someone needing it next week could use right now.

The Forecast page's sorting — same logic, just displayed

The Inventory Forecast report shows this exact queue, visually. The sort order, top to bottom:

1. **Already-holding-bread rows**, sorted by **delivery date** (not ticket number) — because once you HAVE the loaf, the only interesting question left is *when do you need it by*, not who's next in line.
 - If an order's demand got split (some now, some later), the "later" leftover row is glued directly underneath its own "now" row, no matter what.
2. **Force-Reserved-with-no-real-ticket** orders — float to the very top of the not-yet-holding group.
3. **Genuinely ticketed (paid) but not-yet-holding** orders, sorted by **ticket number** (clearance timestamp) — earliest first, exactly like the real engine will actually serve them.
4. **Scheduled Far Out** orders — below everyone actually competing (since they can't win anything right now), but still above:
5. **No-ticket (unpaid) orders** — dead last, since they have no claim on anything at all, ever,

regardless of scheduling.

The newest piece: "what if I confirmed this right now?"

While a quotation is still being *built* — before it's even confirmed, no ticket exists yet — a live "Reserved" number shows next to Quantity on the sale order line. It answers: *"if I hit Confirm this exact second, how much would I actually walk away with, given who's already ahead of me in line?"* It's a pretend-run of the exact same release-and-redeal logic above, using "right now" as the hypothetical ticket time, checking real on-hand stock first and then future truck deliveries — so it never shows a rosier picture than the real engine would actually deliver once the order is confirmed for real.